DÉMO 005-B · CHANGEMENTS ENTRE LES ÉTAPES

Redux puis Redux Toolkit

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-basic (4 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-multi-reducers (2 fichiers)
3. **Étape 2 → 3** · etape-3-provider (7 fichiers)
4. **Étape 3 → 4** · etape-4-payload (5 fichiers)
5. **Étape 4 → 5** · etape-5-middleware (6 fichiers)
6. **Étape 5 → 6** · etape-6-template (10 fichiers)
7. **Étape 6 → 7** · etape-7-redux-toolkit (11 fichiers)

Étape 1 etape-1-basic : point de départ

createStore et un seul reducer : BUY_PHONE, BUY_TV, BUY_CAMERA ; getState, dispatch, subscribe

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (67 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : react-redux ^9.2.0, redux ^5.0.1

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Home from "./pages/Home.jsx";

function App() {
  return <Home />;
}

export default App;
```

DÉPART **src/pages/Home.jsx** 60 lignes

```jsx
import phone from "../assets/phone.png";
import tv from "../assets/tv.png";
import camera from "../assets/camera.png";

import { buyPhone, buyTV, buyCamera, store } from "../store/index.js";
function Home() {
  // getState lit le state une fois : l'écran ne se met pas à jour tout seul (d'où le subscribe)
  const state = store.getState();
  return (
    <div>
```

```
      <h2>{"Welcome to our Application"}</h2>
      <div className="container">
        <div className="item">
          <img src={phone} alt="" width={100} height={100} />
          <div className="availability" id="phones">
            {state.phones} available
          </div>
          <button
            className="button"
            onClick={() => {
              store.dispatch(buyPhone());
            }}
          >
            Buy
          </button>
        </div>
        <div className="item">
          <img src={tv} alt="" width={100} height={100} />
          <div className="availability" id="tvs">
            {state.tvs} available
          </div>
          <button
            className="button"
            onClick={() => {
              store.dispatch(buyTV());
            }}
          >
            Buy
          </button>
        </div>
        <div className="item">
          <img src={camera} alt="" width={100} height={100} />
          <div className="availability" id="cameras">
            {state.cameras} available
          </div>
          <button
            className="button"
            onClick={() => {
              store.dispatch(buyCamera());
            }}
          >
            Buy
          </button>
        </div>
      </div>
    </div>
  );
}

export default Home;
```

DÉPART **src/store/index.js** 73 lignes

```
import { createStore } from "redux";

// Action Types

const BUY_PHONE = "BUY_PHONE";
const BUY_TV = "BUY_TV";
const BUY_CAMERA = "BUY_CAMERA";

const buyPhone = () => {
  return {
    type: BUY_PHONE,
  };
```

```
};

const buyTV = () => {
  return {
    type: BUY_TV,
  };
};

const buyCamera = () => {
  return {
    type: BUY_CAMERA,
  };
};

// reducer

// initial state
const initialState = {
  phones: 5,
  tvs: 3,
  cameras: 8,
};

// (previousState, action) => newState
const phoneReducer = (state = initialState, action) => {
  switch (action.type) {
    case BUY_PHONE:
      return {
        ...state,
        phones: state.phones - 1,
      };
    case BUY_TV:
      return {
        ...state,
        tvs: state.tvs - 1,
      };
    case BUY_CAMERA:
      return {
        ...state,
        cameras: state.cameras - 1,
      };
    default:
      return state;
  }
};

// store
// createStore : le store garde tout le state ; le reducer dit comment il change
const store = createStore(phoneReducer);

// subscribe : cette fonction est appelée après chaque dispatch
store.subscribe(() => {
  const state = store.getState();
  console.log("Updated State:", state);
  // Update the UI elements directly (Test)
  document.getElementById("phones").innerHTML = `${state.phones} available`;
  document.getElementById("tvs").innerHTML = `${state.tvs} available`;
  document.getElementById("cameras").innerHTML = `${state.cameras} available`;
});

export { store, buyPhone, buyTV, buyCamera, phoneReducer };
```

Étape 1 → 2 etape-1-basic → etape-2-multi-reducers

Un reducer par produit, combinés avec combineReducers

MODIFIÉ `src/pages/Home.jsx` +11 −10

```
@@ -1,19 +1,20 @@
- import phone from "../assets/phone.png";
- import tv from "../assets/tv.png";
- import camera from "../assets/camera.png";
-
+ import phoneImg from "../assets/phone.png";
+ import tvImg from "../assets/tv.png";
+ import cameraImg from "../assets/camera.png";
  import { buyPhone, buyTV, buyCamera, store } from "../store/index.js";
  function Home() {
    // getState lit le state une fois : l'écran ne se met pas à jour tout seul (d'où le subscribe)
    const state = store.getState();
+   const { phone, tv, camera } = state;
+
    return (
      <div>
        <h2>{"Welcome to our Application"}</h2>
        <div className="container">
          <div className="item">
-           <img src={phone} alt="" width={100} height={100} />
+           <img src={phoneImg} alt="" width={100} height={100} />
            <div className="availability" id="phones">
-             {state.phones} available
+             {phone.total} available
            </div>
            <button
              className="button"
@@ -25,9 +26,9 @@
            </button>
          </div>
          <div className="item">
-           <img src={tv} alt="" width={100} height={100} />
+           <img src={tvImg} alt="" width={100} height={100} />
            <div className="availability" id="tvs">
-             {state.tvs} available
+             {tv.total} available
            </div>
            <button
              className="button"
@@ -39,9 +40,9 @@
            </button>
          </div>
          <div className="item">
-           <img src={camera} alt="" width={100} height={100} />
+           <img src={cameraImg} alt="" width={100} height={100} />
            <div className="availability" id="cameras">
-             {state.cameras} available
+             {camera.total} available
            </div>
            <button
              className="button"
```

MODIFIÉ `src/store/index.js` +51 −23

```
@@ -1,4 +1,4 @@
- import { createStore } from "redux";
```

```
+ import { combineReducers, createStore } from "redux";
  // Action Types
@@ -27,47 +27,75 @@
  // reducer
  // initial state
- const initialState = {
-   phones: 5,
-   tvs: 3,
-   cameras: 8,
+ const initialStatePhone = {
+   total: 5,
+ };
+
+ const initialStateTV = {
+   total: 3,
+ };
+
+ const initialStateCamera = {
+   total: 8,
  };

  // (previousState, action) => newState
- const phoneReducer = (state = initialState, action) => {
+ const phoneReducer = (state = initialStatePhone, action) => {
    switch (action.type) {
      case BUY_PHONE:
        return {
          ...state,
-         phones: state.phones - 1,
        };
-     case BUY_TV:
-       return {
-         ...state,
-         tvs: state.tvs - 1,
-       };
-     case BUY_CAMERA:
-       return {
-         ...state,
-         cameras: state.cameras - 1,
+         total: state.total - 1,
        };
      default:
        return state;
    }
  };

- // store
- // createStore : le store garde tout le state ; le reducer dit comment il change
- const store = createStore(phoneReducer);
+ // (previousState, action) => newState
+ const tvsReducer = (state = initialStateTV, action) => {
+   switch (action.type) {
+     case BUY_TV:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
```

```
+ // (previousState, action) => newState
+ const camerasReducer = (state = initialStateCamera, action) => {
+   switch (action.type) {
+     case BUY_CAMERA:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
+ const rootReducer = combineReducers({
+   phone: phoneReducer,
+   tv: tvsReducer,
+   camera: camerasReducer,
+ });
+
+ const store = createStore(rootReducer);

  // subscribe : cette fonction est appelée après chaque dispatch
  store.subscribe(() => {
    const state = store.getState();
+   const { phone, tv, camera } = state;
    console.log("Updated State:", state);
    // Update the UI elements directly (Test)
-   document.getElementById("phones").innerHTML = `${state.phones} available`;
-   document.getElementById("tvs").innerHTML = `${state.tvs} available`;
-   document.getElementById("cameras").innerHTML = `${state.cameras} available`;
+   document.getElementById("phones").innerHTML = `${phone.total} available`;
+   document.getElementById("tvs").innerHTML = `${tv.total} available`;
+   document.getElementById("cameras").innerHTML = `${camera.total} available`;
  });

  export { store, buyPhone, buyTV, buyCamera, phoneReducer };
```

Étape 2 → 3 etape-2-multi-reducers → etape-3-provider

Dossier store/ (actions, reducers, store), <Provider>, useSelector, useDispatch

MODIFIÉ `src/App.jsx` +9 −2

```
@@ -1,8 +1,15 @@
  import "./App.css";
- import Home from "./pages/Home.jsx";
+ import Home from "./pages/Home.tsx";
+ import { Provider } from "react-redux";
+ import { store } from "./store/store.ts";

  function App() {
-   return <Home />;
+   // Provider donne le store à tous les composants (useSelector, useDispatch)
+   return (
+     <Provider store={store}>
+       <Home />
+     </Provider>
+   );
  }

  export default App;
```

SUPPRIMÉ `src/pages/Home.jsx` +0 −61

```
@@ -1,61 +1,0 @@
- import phoneImg from "../assets/phone.png";
- import tvImg from "../assets/tv.png";
- import cameraImg from "../assets/camera.png";
- import { buyPhone, buyTV, buyCamera, store } from "../store/index.js";
- function Home() {
-   // getState lit le state une fois : l'écran ne se met pas à jour tout seul (d'où le subscribe)
-   const state = store.getState();
-   const { phone, tv, camera } = state;
-
-   return (
-     <div>
-       <h2>{"Welcome to our Application"}</h2>
-       <div className="container">
-         <div className="item">
-           <img src={phoneImg} alt="" width={100} height={100} />
-           <div className="availability" id="phones">
-             {phone.total} available
-           </div>
-           <button
-             className="button"
-             onClick={() => {
-               store.dispatch(buyPhone());
-             }}
-           >
-             Buy
-           </button>
-         </div>
-         <div className="item">
-           <img src={tvImg} alt="" width={100} height={100} />
-           <div className="availability" id="tvs">
-             {tv.total} available
-           </div>
-           <button
-             className="button"
-             onClick={() => {
```

```
36 -              store.dispatch(buyTV());
37 -            }}
38 -          >
39 -            Buy
40 -          </button>
41 -        </div>
42 -        <div className="item">
43 -          <img src={cameraImg} alt="" width={100} height={100} />
44 -          <div className="availability" id="cameras">
45 -            {camera.total} available
46 -          </div>
47 -          <button
48 -            className="button"
49 -            onClick={() => {
50 -              store.dispatch(buyCamera());
51 -            }}
52 -          >
53 -            Buy
54 -          </button>
55 -        </div>
56 -      </div>
57 -    </div>
58 -  );
59 - }
60 -
61 - export default Home;
```

AJOUTÉ **src/pages/Home.tsx** +70 −0

```
@@ -1,0 +1,70 @@
1  + // @ts-ignore
2  + import phoneImg from "../assets/phone.png";
3  + // @ts-ignore
4  + import tvImg from "../assets/tv.png";
5  + // @ts-ignore
6  + import cameraImg from "../assets/camera.png";
7  +
8  + import { store } from "../store/store";
9  + import { buyPhone, buyTV, buyCamera } from "../store/actions";
10 + import { useSelector, useDispatch } from "react-redux";
11 +
12 + function Home() {
13 +   // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
14 +   const dispatch = useDispatch();
15 +   const phone = useSelector((state: any) => state.phone);
16 +   const camera = useSelector((state: any) => state.camera);
17 +   const tv = useSelector((state: any) => state.tv);
18 +
19 +   return (
20 +     <div>
21 +       <h2>{"Welcome to our Application"}</h2>
22 +       <div className="container">
23 +         <div className="item">
24 +           <img src={phoneImg} alt="" width={100} height={100} />
25 +           <div className="availability" id="phones">
26 +             {phone.total} available
27 +           </div>
28 +           <button
29 +             className="button"
30 +             onClick={() => {
31 +               dispatch(buyPhone());
32 +             }}
33 +           >
34 +             Buy
35 +           </button>
```

```
+         </div>
+         <div className="item">
+           <img src={tvImg} alt="" width={100} height={100} />
+           <div className="availability" id="tvs">
+             {tv.total} available
+           </div>
+           <button
+             className="button"
+             onClick={() => {
+               dispatch(buyTV());
+             }}
+           >
+             Buy
+           </button>
+         </div>
+         <div className="item">
+           <img src={cameraImg} alt="" width={100} height={100} />
+           <div className="availability" id="cameras">
+             {camera.total} available
+           </div>
+           <button
+             className="button"
+             onClick={() => {
+               dispatch(buyCamera());
+             }}
+           >
+             Buy
+           </button>
+         </div>
+       </div>
+     </div>
+   );
+ }
+
+ export default Home;
```

AJOUTÉ **src/store/actions.ts** +25 −0

```
@@ -1,0 +1,25 @@
+ // Action Types
+
+ const BUY_PHONE = "BUY_PHONE";
+ const BUY_TV = "BUY_TV";
+ const BUY_CAMERA = "BUY_CAMERA";
+
+ const buyPhone = () => {
+   return {
+     type: BUY_PHONE,
+   };
+ };
+
+ const buyTV = () => {
+   return {
+     type: BUY_TV,
+   };
+ };
+
+ const buyCamera = () => {
+   return {
+     type: BUY_CAMERA,
+   };
+ };
+
+ export { BUY_PHONE, BUY_TV, BUY_CAMERA, buyPhone, buyTV, buyCamera };
```

SUPPRIMÉ **src/store/index.js** +0 −101

@@ −1,101 +1,0 @@

```
- import { combineReducers, createStore } from "redux";
-
- // Action Types
-
- const BUY_PHONE = "BUY_PHONE";
- const BUY_TV = "BUY_TV";
- const BUY_CAMERA = "BUY_CAMERA";
-
- const buyPhone = () => {
-   return {
-     type: BUY_PHONE,
-   };
- };
-
- const buyTV = () => {
-   return {
-     type: BUY_TV,
-   };
- };
-
- const buyCamera = () => {
-   return {
-     type: BUY_CAMERA,
-   };
- };
-
- // reducer
-
- // initial state
- const initialStatePhone = {
-   total: 5,
- };
-
- const initialStateTV = {
-   total: 3,
- };
-
- const initialStateCamera = {
-   total: 8,
- };
-
- // (previousState, action) => newState
- const phoneReducer = (state = initialStatePhone, action) => {
-   switch (action.type) {
-     case BUY_PHONE:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
-     default:
-       return state;
-   }
- };
-
- // (previousState, action) => newState
- const tvsReducer = (state = initialStateTV, action) => {
-   switch (action.type) {
-     case BUY_TV:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
```

```
63  -     default:
64  -       return state;
65  -   }
66  - };
67  -
68  - // (previousState, action) => newState
69  - const camerasReducer = (state = initialStateCamera, action) => {
70  -   switch (action.type) {
71  -     case BUY_CAMERA:
72  -       return {
73  -         ...state,
74  -         total: state.total - 1,
75  -       };
76  -     default:
77  -       return state;
78  -   }
79  - };
80  -
81  - // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.cam
    era)
82  - const rootReducer = combineReducers({
83  -   phone: phoneReducer,
84  -   tv: tvsReducer,
85  -   camera: camerasReducer,
86  - });
87  -
88  - const store = createStore(rootReducer);
89  -
90  - // subscribe : cette fonction est appelée après chaque dispatch
91  - store.subscribe(() => {
92  -   const state = store.getState();
93  -   const { phone, tv, camera } = state;
94  -   console.log("Updated State:", state);
95  -   // Update the UI elements directly (Test)
96  -   document.getElementById("phones").innerHTML = `${phone.total} available`;
97  -   document.getElementById("tvs").innerHTML = `${tv.total} available`;
98  -   document.getElementById("cameras").innerHTML = `${camera.total} available`;
99  - });
100 -
101 - export { store, buyPhone, buyTV, buyCamera, phoneReducer };
```

AJOUTÉ **src/store/reducers.ts** +57 −0

```
    @@ -1,0 +1,57 @@
1   + // reducer
2   +
3   + import { BUY_CAMERA, BUY_PHONE, BUY_TV } from "./actions";
4   +
5   + // initial state
6   + const initialStatePhone = {
7   +   total: 5,
8   + };
9   +
10  + const initialStateTV = {
11  +   total: 3,
12  + };
13  +
14  + const initialStateCamera = {
15  +   total: 8,
16  + };
17  +
18  + // (previousState, action) => newState
19  + const phoneReducer = (state = initialStatePhone, action) => {
20  +   switch (action.type) {
21  +     case BUY_PHONE:
```

```
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ // (previousState, action) => newState
+ const tvsReducer = (state = initialStateTV, action) => {
+   switch (action.type) {
+     case BUY_TV:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ // (previousState, action) => newState
+ const camerasReducer = (state = initialStateCamera, action) => {
+   switch (action.type) {
+     case BUY_CAMERA:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ export { phoneReducer, tvsReducer, camerasReducer };
```

AJOUTÉ **src/store/store.ts** +21 −0

```
@@ -1,0 +1,21 @@
+ import { combineReducers, createStore, applyMiddleware } from "redux";
+ import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
+
+ // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
+ const rootReducer = combineReducers({
+   phone: phoneReducer,
+   tv: tvsReducer,
+   camera: camerasReducer,
+ });
+
+ // Simple logging middleware
+ const loggerMiddleware = (store) => (next) => (action) => {
+   console.log("Action:", action);
+   const result = next(action);
+   console.log("State after action:", store.getState());
+   return result;
+ };
+
+ const store = createStore(rootReducer, applyMiddleware(loggerMiddleware));
+
+ export { store };
```

Étape 3 → 4 etape-3-provider → etape-4-payload

Action avec payload : choisir la quantité de téléphones

MODIFIÉ `src/App.css` +13 −0

```
@@ -65,3 +65,16 @@
    font-weight: 600;
    color: white;
  }
+
+ .quantity {
+   margin: 0;
+ }
+
+ .input {
+   width: 60px;
+   padding: 4px;
+   font-size: 16px;
+   border: 1px solid #ccc;
+   border-radius: 4px;
+   text-align: center;
+ }
```

MODIFIÉ `src/pages/Home.tsx` +39 −2

```
@@ -5,16 +5,23 @@
  // @ts-ignore
  import cameraImg from "../assets/camera.png";

- import { store } from "../store/store";
  import { buyPhone, buyTV, buyCamera } from "../store/actions";
  import { useSelector, useDispatch } from "react-redux";
+ import { useState } from "react";

  function Home() {
+   const [quantity, setQuantity] = useState(1);
+
    // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
    const dispatch = useDispatch();
    const phone = useSelector((state: any) => state.phone);
    const camera = useSelector((state: any) => state.camera);
    const tv = useSelector((state: any) => state.tv);

+
+   const handleChange = (e) => {
+     const value = e.target.value;
+     setQuantity(value);
+   };

    return (
      <div>
@@ -25,10 +32,20 @@
            <div className="availability" id="phones">
              {phone.total} available
            </div>
+           <div className="quantity">
+             <input
+               className="input"
+               type="number"
+               min="1"
+               max={phone.total}
+               defaultValue={1}
+               onChange={handleChange}
```

```
            />
          </div>
          <button
            className="button"
            onClick={() => {
-             dispatch(buyPhone());
+             dispatch(buyPhone(quantity));
            }}
          >
            Buy
@@ -38,6 +55,16 @@
          <img src={tvImg} alt="" width={100} height={100} />
          <div className="availability" id="tvs">
            {tv.total} available
+         </div>
+         <div className="quantity">
+           <input
+             className="input"
+             type="number"
+             min="1"
+             max={phone.total}
+             defaultValue={1}
+             disabled
+           />
          </div>
          <button
            className="button"
@@ -52,6 +79,16 @@
          <img src={cameraImg} alt="" width={100} height={100} />
          <div className="availability" id="cameras">
            {camera.total} available
+         </div>
+         <div className="quantity">
+           <input
+             className="input"
+             type="number"
+             min="1"
+             max={phone.total}
+             defaultValue={1}
+             disabled
+           />
          </div>
          <button
            className="button"
```

MODIFIÉ **src/store/actions.ts** +3 −1

```
@@ -4,9 +4,11 @@
 const BUY_TV = "BUY_TV";
 const BUY_CAMERA = "BUY_CAMERA";

-const buyPhone = () => {
+const buyPhone = (quantity) => {
   return {
     type: BUY_PHONE,
+    // payload : la donnée envoyée avec l'action (ici la quantité choisie)
+    payload: quantity,
   };
 };
```

MODIFIÉ **src/store/reducers.ts** +1 −1

```
@@ -21,7 +21,7 @@
```

```
21  21        case BUY_PHONE:
22  22          return {
23  23            ...state,
24      -         total: state.total - 1,
    24  +         total: state.total - action.payload,
25  25          };
26  26        default:
27  27          return state;
```

MODIFIÉ **src/store/store.ts** +6 −9

```
    @@ -1,4 +1,4 @@
1       - import { combineReducers, createStore, applyMiddleware } from "redux";
    1   + import { combineReducers, createStore } from "redux";
2   2     import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
3   3
4   4     // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
    @@ -8,14 +8,11 @@
8   8       camera: camerasReducer,
9   9     });
10  10
11      - // Simple logging middleware
12      - const loggerMiddleware = (store) => (next) => (action) => {
13      -   console.log("Action:", action);
14      -   const result = next(action);
15      -   console.log("State after action:", store.getState());
16      -   return result;
17      - };
    11  + const store = createStore(rootReducer);
18  12
19      - const store = createStore(rootReducer, applyMiddleware(loggerMiddleware));
    13  + // subscribe : cette fonction est appelée après chaque dispatch
    14  + store.subscribe(() => {
    15  +   console.log("Updated State:", store.getState());
    16  + });
20  17
21  18    export { store };
```

Étape 4 → 5 etape-4-payload → etape-5-middleware

Middleware : redux-thunk + axios pour charger les commentaires (loading / success / error)

MODIFIÉ **package.json** +3 −1

```
@@ -10,10 +10,12 @@
    "preview": "vite preview"
  },
  "dependencies": {
+   "axios": "^1.13.2",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-redux": "^9.2.0",
-   "redux": "^5.0.1"
+   "redux": "^5.0.1",
+   "redux-thunk": "^3.1.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
```

MODIFIÉ **src/App.css** +35 −0

```
@@ -78,3 +78,38 @@
    border-radius: 4px;
    text-align: center;
  }
+
+ .page {
+   box-sizing: border-box;
+   place-items: center;
+ }
+
+ .comments-section {
+   padding: 12px;
+   width: 610px;
+   border: 0.5px solid mediumslateblue;
+   background-color: beige;
+   box-sizing: border-box;
+   border-radius: 4px;
+ }
+
+ .comment {
+   padding: 12px;
+   border: 1px solid #eee;
+   border-radius: 4px;
+   margin-bottom: 12px;
+   background-color: lightyellow;
+   border: none;
+ }
+
+ .comment-name {
+   font-weight: 600;
+   margin-bottom: 8px;
+   text-align: left;
+ }
+
+ .comment-body {
+   font-weight: 400;
+   white-space: pre-wrap;
+   text-align: left;
+ }
```

MODIFIÉ **src/pages/Home.tsx** +103 −70

```
@@ -7,7 +7,8 @@
 7   7
 8   8   import { buyPhone, buyTV, buyCamera } from "../store/actions";
 9   9   import { useSelector, useDispatch } from "react-redux";
10       - import { useState } from "react";
     10  + import { useEffect, useState } from "react";
     11  + import { apiCall } from "../store/comments/actions";
11  12
12  13   function Home() {
13  14     const [quantity, setQuantity] = useState(1);
@@ -17,87 +18,119 @@
17  18     const phone = useSelector((state: any) => state.phone);
18  19     const camera = useSelector((state: any) => state.camera);
19  20     const tv = useSelector((state: any) => state.tv);
     21  +   const comments = useSelector((state: any) => state.comments);
20  22
21  23     const handleChange = (e) => {
22  24       const value = e.target.value;
23  25       setQuantity(value);
24  26     };
25  27
     28  +   useEffect(() => {
     29  +     // Utiliser l'action thunk pour charger les commentaires
     30  +     dispatch(apiCall());
     31  +   }, []);
     32  +
26  33     return (
27  34       <div>
28  35         <h2>{"Welcome to our Application"}</h2>
29       -       <div className="container">
30       -         <div className="item">
31       -           <img src={phoneImg} alt="" width={100} height={100} />
32       -           <div className="availability" id="phones">
33       -             {phone.total} available
     36  +       <div className="page">
     37  +         <div className="container">
     38  +           <div className="item">
     39  +             <img src={phoneImg} alt="" width={100} height={100} />
     40  +             <div className="availability" id="phones">
     41  +               {phone.total} available
     42  +             </div>
     43  +             <div className="quantity">
     44  +               <input
     45  +                 className="input"
     46  +                 type="number"
     47  +                 min="1"
     48  +                 max={phone.total}
     49  +                 defaultValue={1}
     50  +                 onChange={handleChange}
     51  +               />
     52  +             </div>
     53  +             <button
     54  +               className="button"
     55  +               onClick={() => {
     56  +                 dispatch(buyPhone(quantity));
     57  +               }}
     58  +             >
     59  +               Buy
     60  +             </button>
34  61             </div>
35       -           <div className="quantity">
36       -             <input
```

```
37      -               className="input"
38      -               type="number"
39      -               min="1"
40      -               max={phone.total}
41      -               defaultValue={1}
42      -               onChange={handleChange}
43      -             />
        62  +           <div className="item">
        63  +             <img src={tvImg} alt="" width={100} height={100} />
        64  +             <div className="availability" id="tvs">
        65  +               {tv.total} available
        66  +             </div>
        67  +             <div className="quantity">
        68  +               <input
        69  +                 className="input"
        70  +                 type="number"
        71  +                 min="1"
        72  +                 max={phone.total}
        73  +                 defaultValue={1}
        74  +                 disabled
        75  +               />
        76  +             </div>
        77  +             <button
        78  +               className="button"
        79  +               onClick={() => {
        80  +                 dispatch(buyTV());
        81  +               }}
        82  +             >
        83  +               Buy
        84  +             </button>
44      85              </div>
45      -           <button
46      -             className="button"
47      -             onClick={() => {
48      -               dispatch(buyPhone(quantity));
49      -             }}
50      -           >
51      -             Buy
52      -           </button>
        86  +           <div className="item">
        87  +             <img src={cameraImg} alt="" width={100} height={100} />
        88  +             <div className="availability" id="cameras">
        89  +               {camera.total} available
        90  +             </div>
        91  +             <div className="quantity">
        92  +               <input
        93  +                 className="input"
        94  +                 type="number"
        95  +                 min="1"
        96  +                 max={phone.total}
        97  +                 defaultValue={1}
        98  +                 disabled
        99  +               />
       100  +             </div>
       101  +             <button
       102  +               className="button"
       103  +               onClick={() => {
       104  +                 dispatch(buyCamera());
       105  +               }}
       106  +             >
       107  +               Buy
       108  +             </button>
       109  +           </div>
53     110            </div>
54      -           <div className="item">
```

```
55  -           <img src={tvImg} alt="" width={100} height={100} />
56  -           <div className="availability" id="tvs">
57  -             {tv.total} available
58  -           </div>
59  -           <div className="quantity">
60  -             <input
61  -               className="input"
62  -               type="number"
63  -               min="1"
64  -               max={phone.total}
65  -               defaultValue={1}
66  -               disabled
67  -             />
68  -           </div>
69  -           <button
70  -             className="button"
71  -             onClick={() => {
72  -               dispatch(buyTV());
73  -             }}
74  -           >
75  -             Buy
76  -           </button>
77  -         </div>
78  -         <div className="item">
79  -           <img src={cameraImg} alt="" width={100} height={100} />
80  -           <div className="availability" id="cameras">
81  -             {camera.total} available
82  -           </div>
83  -           <div className="quantity">
84  -             <input
85  -               className="input"
86  -               type="number"
87  -               min="1"
88  -               max={phone.total}
89  -               defaultValue={1}
90  -               disabled
91  -             />
92  -           </div>
93  -           <button
94  -             className="button"
95  -             onClick={() => {
96  -               dispatch(buyCamera());
97  -             }}
98  -           >
99  -             Buy
100 -           </button>
111 +         <div className="comments-section">
112 +           {comments.isLoading ? (
113 +             <div>Chargement des commentaires...</div>
114 +           ) : comments.error ? (
115 +             <div>Erreur: {comments.error}</div>
116 +           ) : (
117 +             <div className="comments-list">
118 +               {comments.comments.length > 0 ? (
119 +                 comments.comments
120 +                   .slice(0, 10)
121 +                   .map((comment: any, index: number) => (
122 +                     <div key={comment.id || index} className="comment">
123 +                       <div className="comment-name">
124 +                         {comment.name || comment.email}
125 +                       </div>
126 +                       <div className="comment-body">{comment.body}</div>
127 +                     </div>
128 +                   ))
129 +               ) : (
```

```
                <div>Aucun commentaire disponible</div>
              )}
            </div>
          )}
        </div>
      </div>
    </div>
```

AJOUTÉ **src/store/comments/actions.ts** +41 −0

```
@@ -1,0 +1,41 @@
+ export const LOAD_COMMENTS = "LOAD_COMMENTS";
+ export const LOAD_COMMENTS_SUCCESS = "LOAD_COMMENTS_SUCCESS";
+ export const LOAD_COMMENTS_ERROR = "LOAD_COMMENTS_ERROR";
+
+ import axios from "axios";
+
+ const loadApiComments = () => {
+   return {
+     type: LOAD_COMMENTS,
+   };
+ };
+
+ const loadCommentsSuccess = (comments) => {
+   return {
+     type: LOAD_COMMENTS_SUCCESS,
+     payload: comments,
+   };
+ };
+
+ const loadCommentsError = (error) => {
+   return {
+     type: LOAD_COMMENTS_ERROR,
+     payload: error,
+   };
+ };
+
+ // Avec redux-thunk, une action peut être une fonction : elle reçoit dispatch et fait l'appel API
+ export const apiCall = () => {
+   return async (dispatch) => {
+     dispatch(loadApiComments());
+
+     try {
+       const response = await axios.get(
+         "https://jsonplaceholder.typicode.com/comments?postId=1"
+       );
+       dispatch(loadCommentsSuccess(response.data));
+     } catch (error) {
+       dispatch(loadCommentsError(error.message));
+     }
+   };
+ };
```

AJOUTÉ **src/store/comments/reducers.ts** +40 −0

```
@@ -1,0 +1,40 @@
+ import {
+   LOAD_COMMENTS,
+   LOAD_COMMENTS_ERROR,
+   LOAD_COMMENTS_SUCCESS,
+ } from "./actions";
+
+ const initialStateComments = {
+   isLoading: false,
```

```
+   comments: [],
+   error: "",
+ };
+
+ const commentReducer = (state = initialStateComments, action) => {
+   switch (action.type) {
+     case LOAD_COMMENTS:
+       return {
+         ...state,
+         isLoading: true,
+       };
+     case LOAD_COMMENTS_SUCCESS:
+       return {
+         ...state,
+         isLoading: false,
+         comments: action.payload,
+         error: "",
+       };
+     case LOAD_COMMENTS_ERROR:
+       return {
+         ...state,
+         isLoading: false,
+         comments: [],
+         error: action.payload,
+       };
+
+     default:
+       return state;
+   }
+ };
+
+ export default commentReducer;
```

MODIFIÉ **src/store/store.ts** +16 −2

```
@@ -1,14 +1,28 @@
- import { combineReducers, createStore } from "redux";
+ import { applyMiddleware, combineReducers, createStore, compose } from "redux";
  import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
+ import commentReducer from "./comments/reducers";
+ // Import nommé au lieu de default
+ import { thunk } from "redux-thunk";

  // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
  const rootReducer = combineReducers({
    phone: phoneReducer,
    tv: tvsReducer,
    camera: camerasReducer,
+   comments: commentReducer,
  });

- const store = createStore(rootReducer);
+ // applyMiddleware(thunk) : permet de dispatcher des fonctions (apiCall), pas seulement des objets
+ const store = createStore(rootReducer, applyMiddleware(thunk));
+
+ /*
+ const composeEnhancers = window.__REDUX_DEVTOOLS_EXTENSION_COMPOSE__ || compose;
+
+ const store = createStore(
+   rootReducer,
+   composeEnhancers(applyMiddleware(thunk))
+ );
+ */
```

```
12  26
13  27  // subscribe : cette fonction est appelée après chaque dispatch
14  28  store.subscribe(() => {
```

Étape 5 → 6 etape-5-middleware → etape-6-template

Redux Toolkit : configureStore, un premier createSlice (phone)

MODIFIÉ **package.json** +3 −3

```
@@ -10,12 +10,12 @@
    "preview": "vite preview"
  },
  "dependencies": {
-   "axios": "^1.13.2",
+   "@reduxjs/toolkit": "^2.11.2",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-redux": "^9.2.0",
-   "redux": "^5.0.1",
-   "redux-thunk": "^3.1.0"
+   "react-router-dom": "^7.11.0",
+   "redux": "^5.0.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
```

MODIFIÉ **src/App.css** +0 −35

```
@@ -78,38 +78,3 @@
    border-radius: 4px;
    text-align: center;
  }
-
- .page {
-   box-sizing: border-box;
-   place-items: center;
- }
-
- .comments-section {
-   padding: 12px;
-   width: 610px;
-   border: 0.5px solid mediumslateblue;
-   background-color: beige;
-   box-sizing: border-box;
-   border-radius: 4px;
- }
-
- .comment {
-   padding: 12px;
-   border: 1px solid #eee;
-   border-radius: 4px;
-   margin-bottom: 12px;
-   background-color: lightyellow;
-   border: none;
- }
-
- .comment-name {
-   font-weight: 600;
-   margin-bottom: 8px;
-   text-align: left;
- }
-
- .comment-body {
-   font-weight: 400;
-   white-space: pre-wrap;
-   text-align: left;
```

```
115 -  }
```

MODIFIÉ **src/App.jsx** +0 −1

```
@@ -2,7 +2,6 @@
2  2    import Home from "./pages/Home.tsx";
3  3    import { Provider } from "react-redux";
4  4    import { store } from "./store/store.ts";
5     -
6  5    function App() {
7  6      // Provider donne le store à tous les composants (useSelector, useDispatch)
8  7      return (
```

MODIFIÉ **src/pages/Home.tsx** +24 −116

```
@@ -5,132 +5,40 @@
5  5    // @ts-ignore
6  6    import cameraImg from "../assets/camera.png";
7  7
8     - import { buyPhone, buyTV, buyCamera } from "../store/actions";
9     - import { useSelector, useDispatch } from "react-redux";
10    - import { useEffect, useState } from "react";
11    - import { apiCall } from "../store/comments/actions";
   8  + // Attention : encore les anciennes actions de actions.ts, pas celles du slice (corrigé à l'étape 7)
   9  + import { buyPhone, buyTv, buyCamera } from "../store/actions";
   10 + import { useDispatch, useSelector } from "react-redux";
12 11
13 12   function Home() {
14    -   const [quantity, setQuantity] = useState(1);
15    -
16 13     // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
17 14     const dispatch = useDispatch();
18 15     const phone = useSelector((state: any) => state.phone);
   16 +   const tv = useSelector((state: any) => state.tv);
19 17     const camera = useSelector((state: any) => state.camera);
20    -   const tv = useSelector((state: any) => state.tv);
21    -   const comments = useSelector((state: any) => state.comments);
22    -
23    -   const handleChange = (e) => {
24    -     const value = e.target.value;
25    -     setQuantity(value);
26    -   };
27    -
28    -   useEffect(() => {
29    -     // Utiliser l'action thunk pour charger les commentaires
30    -     dispatch(apiCall());
31    -   }, []);
32    -
33 18     return (
34 19       <div>
35 20         <h2>{"Welcome to our Application"}</h2>
36    -       <div className="page">
37    -         <div className="container">
38    -           <div className="item">
39    -             <img src={phoneImg} alt="" width={100} height={100} />
40    -             <div className="availability" id="phones">
41    -               {phone.total} available
42    -             </div>
43    -             <div className="quantity">
44    -               <input
45    -                 className="input"
46    -                 type="number"
47    -                 min="1"
48    -                 max={phone.total}
```

```
-                defaultValue={1}
-                onChange={handleChange}
-              />
-            </div>
-            <button
-              className="button"
-              onClick={() => {
-                dispatch(buyPhone(quantity));
-              }}
-            >
-              Buy
-            </button>
-          </div>
-          <div className="item">
-            <img src={tvImg} alt="" width={100} height={100} />
-            <div className="availability" id="tvs">
-              {tv.total} available
-            </div>
-            <div className="quantity">
-              <input
-                className="input"
-                type="number"
-                min="1"
-                max={phone.total}
-                defaultValue={1}
-                disabled
-              />
-            </div>
-            <button
-              className="button"
-              onClick={() => {
-                dispatch(buyTV());
-              }}
-            >
-              Buy
-            </button>
-          </div>
-          <div className="item">
-            <img src={cameraImg} alt="" width={100} height={100} />
-            <div className="availability" id="cameras">
-              {camera.total} available
-            </div>
-            <div className="quantity">
-              <input
-                className="input"
-                type="number"
-                min="1"
-                max={phone.total}
-                defaultValue={1}
-                disabled
-              />
-            </div>
-            <button
-              className="button"
-              onClick={() => {
-                dispatch(buyCamera());
-              }}
-            >
-              Buy
-            </button>
-          </div>
+      <div className="container">
+        <div className="item">
+          <img src={phoneImg} alt="" width={100} height={100} />
+          <div className="availability">{phone.total} available</div>
```

```
                <button className="button" onClick={() => dispatch(buyPhone(2))}>
                  Buy
                </button>
              </div>
-             <div className="comments-section">
-               {comments.isLoading ? (
-                 <div>Chargement des commentaires...</div>
-               ) : comments.error ? (
-                 <div>Erreur: {comments.error}</div>
-               ) : (
-                 <div className="comments-list">
-                   {comments.comments.length > 0 ? (
-                     comments.comments
-                       .slice(0, 10)
-                       .map((comment: any, index: number) => (
-                         <div key={comment.id || index} className="comment">
-                           <div className="comment-name">
-                             {comment.name || comment.email}
-                           </div>
-                           <div className="comment-body">{comment.body}</div>
-                         </div>
-                       ))
-                   ) : (
-                     <div>Aucun commentaire disponible</div>
-                   )}
-                 </div>
-               )}
+             <div className="item">
+               <img src={tvImg} alt="" width={100} height={100} />
+               <div className="availability">{tv.total} available</div>
+               <button className="button" onClick={() => dispatch(buyTv())}>
+                 Buy
+               </button>
+             </div>{" "}
+             <div className="item">
+               <img src={cameraImg} alt="" width={100} height={100} />
+               <div className="availability">{camera.total} available</div>
+               <button className="button" onClick={() => dispatch(buyCamera())}>
+                 Buy
+               </button>
              </div>
            </div>
          </div>
```

MODIFIÉ **src/store/actions.ts** +6 −6

```
@@ -1,8 +1,4 @@
- // Action Types
-
  const BUY_PHONE = "BUY_PHONE";
- const BUY_TV = "BUY_TV";
- const BUY_CAMERA = "BUY_CAMERA";

  const buyPhone = (quantity) => {
    return {
@@ -12,11 +8,15 @@
    };
  };

- const buyTV = () => {
+ const BUY_TV = "BUY_TV";
+
+ const buyTv = () => {
    return {
```

```
17  15        type: BUY_TV,
18  16      };
19  17    };
    18  +
    19  + const BUY_CAMERA = "BUY_CAMERA";
20  20
21  21    const buyCamera = () => {
22  22      return {
          @@ -24,4 +24,4 @@
24  24      };
25  25    };
26  26
27      - export { BUY_PHONE, BUY_TV, BUY_CAMERA, buyPhone, buyTV, buyCamera };
    27  + export { BUY_PHONE, buyPhone, BUY_TV, buyTv, BUY_CAMERA, buyCamera };
```

SUPPRIMÉ **src/store/comments/actions.ts** +0 −41

```
          @@ -1,41 +1,0 @@
1       - export const LOAD_COMMENTS = "LOAD_COMMENTS";
2       - export const LOAD_COMMENTS_SUCCESS = "LOAD_COMMENTS_SUCCESS";
3       - export const LOAD_COMMENTS_ERROR = "LOAD_COMMENTS_ERROR";
4       -
5       - import axios from "axios";
6       -
7       - const loadApiComments = () => {
8       -   return {
9       -     type: LOAD_COMMENTS,
10      -   };
11      - };
12      -
13      - const loadCommentsSuccess = (comments) => {
14      -   return {
15      -     type: LOAD_COMMENTS_SUCCESS,
16      -     payload: comments,
17      -   };
18      - };
19      -
20      - const loadCommentsError = (error) => {
21      -   return {
22      -     type: LOAD_COMMENTS_ERROR,
23      -     payload: error,
24      -   };
25      - };
26      -
27      - // Avec redux-thunk, une action peut être une fonction : elle reçoit dispatch et fait l'app
        el API
28      - export const apiCall = () => {
29      -   return async (dispatch) => {
30      -     dispatch(loadApiComments());
31      -
32      -     try {
33      -       const response = await axios.get(
34      -         "https://jsonplaceholder.typicode.com/comments?postId=1"
35      -       );
36      -       dispatch(loadCommentsSuccess(response.data));
37      -     } catch (error) {
38      -       dispatch(loadCommentsError(error.message));
39      -     }
40      -   };
41      - };
```

SUPPRIMÉ **src/store/comments/reducers.ts** +0 −40

```
          @@ -1,40 +1,0 @@
```

```
- import {
-   LOAD_COMMENTS,
-   LOAD_COMMENTS_ERROR,
-   LOAD_COMMENTS_SUCCESS,
- } from "./actions";
-
- const initialStateComments = {
-   isLoading: false,
-   comments: [],
-   error: "",
- };
-
- const commentReducer = (state = initialStateComments, action) => {
-   switch (action.type) {
-     case LOAD_COMMENTS:
-       return {
-         ...state,
-         isLoading: true,
-       };
-     case LOAD_COMMENTS_SUCCESS:
-       return {
-         ...state,
-         isLoading: false,
-         comments: action.payload,
-         error: "",
-       };
-     case LOAD_COMMENTS_ERROR:
-       return {
-         ...state,
-         isLoading: false,
-         comments: [],
-         error: action.payload,
-       };
-
-     default:
-       return state;
-   }
- };
-
- export default commentReducer;
```

AJOUTÉ **src/store/phoneSlice.ts** +17 −0

```
@@ -1,0 +1,17 @@
+ import { createSlice } from "@reduxjs/toolkit";
+
+ const phoneSlice = createSlice({
+   name: "phone",
+   initialState: {
+     total: 10,
+   },
+   // Chaque reducer devient une action (phoneSlice.actions) ; Immer permet de « modifier » state directement
+   reducers: {
+     buyPhone: (state, action) => {
+       state.total -= action.payload || 1;
+     },
+   },
+ });
+
+ export const { buyPhone } = phoneSlice.actions;
+ export const phoneReducer = phoneSlice.reducer;
```

MODIFIÉ **src/store/reducers.ts** +14 −21

```
@@ -1,22 +1,10 @@
- // reducer
+ import { BUY_PHONE, BUY_TV, BUY_CAMERA } from "./actions";

- import { BUY_CAMERA, BUY_PHONE, BUY_TV } from "./actions";
-
- // initial state
- const initialStatePhone = {
-   total: 5,
+ const phoneInitialState = {
+   total: 10,
  };

- const initialStateTV = {
-   total: 3,
- };
-
- const initialStateCamera = {
-   total: 8,
- };
-
- // (previousState, action) => newState
- const phoneReducer = (state = initialStatePhone, action) => {
+ const phoneReducer = (state = phoneInitialState, action) => {
    switch (action.type) {
      case BUY_PHONE:
        return {
@@ -27,9 +15,11 @@
        return state;
    }
  };
+ const tvInitialState = {
+   total: 8,
+ };

- // (previousState, action) => newState
- const tvsReducer = (state = initialStateTV, action) => {
+ const tvReducer = (state = tvInitialState, action) => {
    switch (action.type) {
      case BUY_TV:
        return {
@@ -41,8 +31,11 @@
    }
  };

- // (previousState, action) => newState
- const camerasReducer = (state = initialStateCamera, action) => {
+ const cameraInitialState = {
+   total: 5,
+ };
+
+ const cameraReducer = (state = cameraInitialState, action) => {
    switch (action.type) {
      case BUY_CAMERA:
        return {
@@ -54,4 +47,4 @@
    }
  };

- export { phoneReducer, tvsReducer, camerasReducer };
+ export { phoneReducer, tvReducer, cameraReducer };
```

MODIFIÉ **src/store/store.ts** +8 −30

```
@@ -1,32 +1,10 @@
- import { applyMiddleware, combineReducers, createStore, compose } from "redux";
- import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
- import commentReducer from "./comments/reducers";
- // Import nommé au lieu de default
- import { thunk } from "redux-thunk";
+ import { configureStore } from "@reduxjs/toolkit";
+ import { phoneReducer } from "./phoneSlice";

- // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
- const rootReducer = combineReducers({
-   phone: phoneReducer,
-   tv: tvsReducer,
-   camera: camerasReducer,
-   comments: commentReducer,
+ // Attention : seul phone est déclaré ; Home lit aussi state.tv et state.camera (ajoutés à l'étape 7)
+ export const store = configureStore({
+   reducer: {
+     phone: phoneReducer,
+   },
+   devTools: true,
  });
-
- // applyMiddleware(thunk) : permet de dispatcher des fonctions (apiCall), pas seulement des objets
- const store = createStore(rootReducer, applyMiddleware(thunk));
-
- /*
- const composeEnhancers = window.__REDUX_DEVTOOLS_EXTENSION_COMPOSE__ || compose;
-
- const store = createStore(
-   rootReducer,
-   composeEnhancers(applyMiddleware(thunk))
- );
- */
-
- // subscribe : cette fonction est appelée après chaque dispatch
- store.subscribe(() => {
-   console.log("Updated State:", store.getState());
- });
-
- export { store };
```

Étape 6 → 7 etape-6-template → etape-7-redux-toolkit

Les 3 slices (phone, tv, camera), extraReducers (promo TV), RTK Query (createApi)

MODIFIÉ `package.json` +0 −1

```
@@ -14,7 +14,6 @@
      "react": "^19.2.0",
      "react-dom": "^19.2.0",
      "react-redux": "^9.2.0",
-     "react-router-dom": "^7.11.0",
      "redux": "^5.0.1"
    },
    "devDependencies": {
```

MODIFIÉ `src/App.css` +0 −13

```
@@ -65,16 +65,3 @@
    font-weight: 600;
    color: white;
  }
-
- .quantity {
-   margin: 0;
- }
-
- .input {
-   width: 60px;
-   padding: 4px;
-   font-size: 16px;
-   border: 1px solid #ccc;
-   border-radius: 4px;
-   text-align: center;
- }
```

MODIFIÉ `src/App.jsx` +1 −0

```
@@ -2,6 +2,7 @@
  import Home from "./pages/Home.tsx";
  import { Provider } from "react-redux";
  import { store } from "./store/store.ts";
+
  function App() {
    // Provider donne le store à tous les composants (useSelector, useDispatch)
    return (
```

AJOUTÉ `src/apis/index.ts` +70 −0

```
@@ -1,0 +1,70 @@
+ // RTK Query (Redux Toolkit Query) : on crée une "API slice"
+ // Objectif : faire des appels HTTP + gérer cache / loading / error automatiquement
+
+ import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
+
+ /**
+  * commentsApi = "API slice"
+  * Elle contient :
+  * - un reducer (pour stocker cache + status dans Redux)
+  * - un middleware (pour déclencher/optimiser les requêtes)
+  * - des endpoints (GET/POST/...)
+  * - des hooks React auto-générés (useGetCommentsQuery, etc.)
+  */
+ export const commentsApi = createApi({
```

```
+   /**
+    * reducerPath = le nom de la clé dans le store Redux
+    * Exemple : state.commentsApi => contiendra le cache, loading, erreurs, etc.
+    *
+    * Dans le store on fera :
+    * reducer: { [commentsApi.reducerPath]: commentsApi.reducer }
+    */
+   reducerPath: "commentsApi",
+
+   /**
+    * baseQuery = le "client HTTP" utilisé par RTK Query
+    * fetchBaseQuery est une petite couche au-dessus de fetch (pas besoin d'axios)
+    *
+    * baseUrl = URL de base, les endpoints ajouteront juste le chemin
+    */
+   baseQuery: fetchBaseQuery({
+     baseUrl: "https://jsonplaceholder.typicode.com",
+   }),
+
+   /**
+    * endpoints = on déclare ici toutes les routes API
+    * builder permet de créer :
+    * - builder.query(...)  => GET (lecture) + cache
+    * - builder.mutation(...) => POST/PUT/DELETE (écriture)
+    */
+   endpoints: (builder) => ({
+     /**
+      * getComments = nom de l'endpoint
+      * builder.query => c'est un endpoint de lecture (GET)
+      */
+     getComments: builder.query({
+       /**
+        * query = fonction qui construit l'URL de la requête
+        * postId est un paramètre (optionnel) avec une valeur par défaut = 1
+        *
+        * Si postId = 1 => GET /comments?postId=1
+        * Si postId = 5 => GET /comments?postId=5
+        *
+        * RTK Query gère automatiquement :
+        * - loading / success / error
+        * - cache + dedupe (évite des appels identiques)
+        */
+       query: (postId = 1) => `/comments?postId=${postId}`,
+     }),
+   }),
+ });
+
+ /**
+  * RTK Query génère automatiquement des hooks React
+  * Le nom est basé sur l'endpoint :
+  * getComments => useGetCommentsQuery
+  *
+  * Utilisation :
+  * const { data, isLoading, error } = useGetCommentsQuery(1);
+  */
+ export const { useGetCommentsQuery } = commentsApi;
```

MODIFIÉ **src/pages/Home.tsx** +39 −11

```
@@ -5,38 +5,66 @@
  // @ts-ignore
  import cameraImg from "../assets/camera.png";

- // Attention : encore les anciennes actions de actions.ts, pas celles du slice (corrigé à l'étape 7)
```

```
 9        - import { buyPhone, buyTv, buyCamera } from "../store/actions";
10        - import { useDispatch, useSelector } from "react-redux";
          8 + import { buyPhone } from "../store/phoneSlice";
          9 + import { buyTv } from "../store/tvSlice";
         10 + import { buyCamera } from "../store/cameraSlice";
         11 +
         12 + import { useSelector, useDispatch } from "react-redux";
         13 + import { useGetCommentsQuery } from "../apis";
11       14
12       15   function Home() {
13       16     // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
14       17     const dispatch = useDispatch();
15       18     const phone = useSelector((state: any) => state.phone);
         19 +   const camera = useSelector((state: any) => state.camera);
16       20     const tv = useSelector((state: any) => state.tv);
17        -   const camera = useSelector((state: any) => state.camera);
         21 +   // Hook généré par RTK Query : il fait la requête et donne data / isLoading / error
         22 +   const { data, isLoading, error } = useGetCommentsQuery(1);
         23 +   console.log({ data, isLoading, error });
         24 +
18       25     return (
19       26       <div>
20       27         <h2>{"Welcome to our Application"}</h2>
21       28         <div className="container">
22       29           <div className="item">
23       30             <img src={phoneImg} alt="" width={100} height={100} />
24        -            <div className="availability">{phone.total} available</div>
25        -            <button className="button" onClick={() => dispatch(buyPhone(2))}>
         31 +            <div className="availability" id="phones">
         32 +              {phone.total} available
         33 +            </div>
         34 +            <button
         35 +              className="button"
         36 +              onClick={() => {
         37 +                dispatch(buyPhone());
         38 +              }}
         39 +            >
26       40               Buy
27       41             </button>
28       42           </div>
29       43           <div className="item">
30       44             <img src={tvImg} alt="" width={100} height={100} />
31        -            <div className="availability">{tv.total} available</div>
32        -            <button className="button" onClick={() => dispatch(buyTv())}>
         45 +            <div className="availability" id="tvs">
         46 +              {tv.total} available
         47 +            </div>
         48 +            <button
         49 +              className="button"
         50 +              onClick={() => {
         51 +                dispatch(buyTv());
         52 +              }}
         53 +            >
33       54               Buy
34       55             </button>
35        -          </div>{" "}
         56 +          </div>
36       57           <div className="item">
37       58             <img src={cameraImg} alt="" width={100} height={100} />
38        -            <div className="availability">{camera.total} available</div>
39        -            <button className="button" onClick={() => dispatch(buyCamera())}>
         59 +            <div className="availability" id="cameras">
         60 +              {camera.total} available
         61 +            </div>
         62 +            <button
```

```
                63 +              className="button"
                64 +              onClick={() => {
                65 +                dispatch(buyCamera());
                66 +              }}
                67 +            >
40              68                Buy
41              69              </button>
42              70            </div>
```

SUPPRIMÉ **src/store/actions.ts** +0 −27

```
                   @@ -1,27 +1,0 @@
 1                 - const BUY_PHONE = "BUY_PHONE";
 2                 -
 3                 - const buyPhone = (quantity) => {
 4                 -   return {
 5                 -     type: BUY_PHONE,
 6                 -     // payload : la donnée envoyée avec l'action (ici la quantité choisie)
 7                 -     payload: quantity,
 8                 -   };
 9                 - };
10                 -
11                 - const BUY_TV = "BUY_TV";
12                 -
13                 - const buyTv = () => {
14                 -   return {
15                 -     type: BUY_TV,
16                 -   };
17                 - };
18                 -
19                 - const BUY_CAMERA = "BUY_CAMERA";
20                 -
21                 - const buyCamera = () => {
22                 -   return {
23                 -     type: BUY_CAMERA,
24                 -   };
25                 - };
26                 -
27                 - export { BUY_PHONE, buyPhone, BUY_TV, buyTv, BUY_CAMERA, buyCamera };
```

AJOUTÉ **src/store/cameraSlice.ts** +17 −0

```
                   @@ -1,0 +1,17 @@
                 1 + // features/stock/cameraSlice.ts
                 2 + import { createSlice } from "@reduxjs/toolkit";
                 3 +
                 4 + const initialState = { total: 8 };
                 5 +
                 6 + const cameraSlice = createSlice({
                 7 +   name: "camera",
                 8 +   initialState,
                 9 +   reducers: {
                10 +     buyCamera(state) {
                11 +       state.total -= 1;
                12 +     },
                13 +   },
                14 + });
                15 +
                16 + export const { buyCamera } = cameraSlice.actions;
                17 + export default cameraSlice.reducer;
```

MODIFIÉ **src/store/phoneSlice.ts** +24 −7

```
                   @@ -1,17 +1,34 @@
 1                 - import { createSlice } from "@reduxjs/toolkit";
```

```
+ // features/stock/phoneSlice.ts
+ import { createSlice, PayloadAction } from "@reduxjs/toolkit";
+ // import { buyTv, buyTvPromotion } from "./tvSlice";
+
+ const initialState = { total: 5 };
  
  const phoneSlice = createSlice({
    name: "phone",
-   initialState: {
-     total: 10,
-   },
+   initialState,
    // Chaque reducer devient une action (phoneSlice.actions) ; Immer permet de « modifier » state directement
    reducers: {
-     buyPhone: (state, action) => {
-       state.total -= action.payload || 1;
+     buyPhone(state) {
+       // action.payload can be used for more complex logic
+       state.total -= 1;
      },
    },
+   // extraReducers (en commentaire) : réagir à une action d'un autre slice, ex. la promo TV
+   /*extraReducers: (builder) => {
+     builder.addCase(buyTv, (state) => {
+       state.total -= 1; // si achat TV => +1 phone
+     });
+   },*/
+   /*
+     extraReducers: (builder) => {
+     builder.addCase(buyTvPromotion, (state, action: PayloadAction<string>) => {
+       if (action.payload === "2025XMAS") {
+         state.total -= 1;
+       }
+     });
+   }, */
  });
  
  export const { buyPhone } = phoneSlice.actions;
- export const phoneReducer = phoneSlice.reducer;
+ export default phoneSlice.reducer;
```

SUPPRIMÉ **src/store/reducers.ts** +0 −50

```
@@ -1,50 +1,0 @@
- import { BUY_PHONE, BUY_TV, BUY_CAMERA } from "./actions";
-
- const phoneInitialState = {
-   total: 10,
- };
-
- const phoneReducer = (state = phoneInitialState, action) => {
-   switch (action.type) {
-     case BUY_PHONE:
-       return {
-         ...state,
-         total: state.total - action.payload,
-       };
-     default:
-       return state;
-   }
- };
- const tvInitialState = {
-   total: 8,
- };
```

```
21  -
22  - const tvReducer = (state = tvInitialState, action) => {
23  -   switch (action.type) {
24  -     case BUY_TV:
25  -       return {
26  -         ...state,
27  -         total: state.total - 1,
28  -       };
29  -     default:
30  -       return state;
31  -   }
32  - };
33  -
34  - const cameraInitialState = {
35  -   total: 5,
36  - };
37  -
38  - const cameraReducer = (state = cameraInitialState, action) => {
39  -   switch (action.type) {
40  -     case BUY_CAMERA:
41  -       return {
42  -         ...state,
43  -         total: state.total - 1,
44  -       };
45  -     default:
46  -       return state;
47  -   }
48  - };
49  -
50  - export { phoneReducer, tvReducer, cameraReducer };
```

MODIFIÉ **src/store/store.ts** +12 −3

```
@@ -1,10 +1,19 @@
    1 + // store.ts
1   2   import { configureStore } from "@reduxjs/toolkit";
2     - import { phoneReducer } from "./phoneSlice";
    3 + import phoneReducer from "./phoneSlice";
    4 + import tvReducer from "./tvSlice";
    5 + import cameraReducer from "./cameraSlice";
    6 + import { commentsApi } from "../apis";
3   7
4     - // Attention : seul phone est déclaré ; Home lit aussi state.tv et state.camera (ajoutés à l'étape 7)
5   8   export const store = configureStore({
6   9     reducer: {
7   10      phone: phoneReducer,
    11 +    tv: tvReducer,
    12 +    camera: cameraReducer,
    13 +    commentsApi: commentsApi.reducer,
8   14    },
9     -   devTools: true,
    15 +   // On ajoute le middleware de RTK Query (requêtes, cache) à ceux par défaut
    16 +   middleware: (getDefaultMiddleware) =>
    17 +     getDefaultMiddleware().concat(commentsApi.middleware),
    18 +   devTools: true, // optionnel (par défaut true en dev)
10  19  });
```

AJOUTÉ **src/store/tvSlice.ts** +20 −0

```
@@ -1,0 +1,20 @@
    1 + // features/stock/tvSlice.ts
    2 + import { createSlice, PayloadAction } from "@reduxjs/toolkit";
    3 +
    4 + const initialState = { total: 3 };
```

```
+
+ const tvSlice = createSlice({
+   name: "tv",
+   initialState,
+   reducers: {
+     buyTv(state) {
+       state.total -= 1;
+     },
+     /*buyTvPromotion(state, action: PayloadAction<string>) {
+       state.total -= 1;
+     }/*, */
+   },
+ });
+
+ export const { buyTv } = tvSlice.actions;
+ export default tvSlice.reducer;
```